



Lab Number: 14

Course Code: CL2005

Course Title: Database Systems - Lab

Semester: Spring 2026

Agenda: MongoDB Data Querying

Instructor: Muhammad Mehdi

Email: muhammad.mehdi@nu.edu.pk

Office: Rafaqat Lab



Table of Contents

1 Data Types in MongoDB.....	1
1.1 Arrays.....	2
1.2 Embedded Documents.....	2
2 Querying Documents.....	3
2.1 Sorting.....	4
2.2 Limiting.....	4
2.3 Skipping.....	5
2.4 Projection.....	5
2.5 Filtering.....	6
2.5.1 Comparison Operators.....	7
2.5.2 Handling Missing Values.....	8
2.5.3 Logical Operators.....	8
2.5.4 Array Filtering.....	9
2.5.4.1 Basic Array Matching.....	10
2.5.4.2 Array Operators.....	10
2.5.5 Querying Embedded Documents.....	11



1 Data Types in MongoDB

MongoDB stores data in **BSON (Binary JSON)** format, which supports a wide range of data types. This allows documents to store simple, complex, and nested data structures in a flexible way.

Unlike relational databases, MongoDB allows **different documents in the same collection to have different structures**, making it more flexible for real-world applications.

Category	Data Types
Numbers	Int32, Int64, Double, Decimal128
Text	String
Logical	Boolean
Empty	Null
Structure	Array, Object (Embedded Document)
Special	ObjectId, Date

Example:

```
{
  _id: ObjectId("507f1f77bcf86cd799439011"),           // ObjectId

  userId: NumberInt(1024),                             // Int32
  accountBalance: NumberLong("15000000000"),           // Int64

  rating: 4.7,                                           // Double
  accountCredit: NumberDecimal("2500.75"),             // Decimal128

  fullName: "Ali Khan",                                // String
  isVerified: true,                                     // Boolean
  middleName: null,                                     // Null

  interests: ["web", "mobile", "desktop"],              // Array

  address: {street: "Phase 1", city: "Peshawar"},        // Embedded Document

  createdAt: new Date("2026-04-26T10:00:00Z")          // Date (ISO-UTC)
}
```



1.1 Arrays

Arrays store **multiple values in a single field**. They are useful when a field can have **more than one value**, such as skills, courses, or tags.

In MongoDB, arrays eliminate the need for separate tables (as used in SQL for multivalued attributes).

Properties:

- Arrays can store mixed data types
- Arrays can be empty
- You can query individual elements easily

General Syntax:

```
{  
  field: [value1, value2, ...]  
}
```

Example:

```
{  
  skills: ["Python", "PHP", "Django", "Laravel"]  
}
```

1.2 Embedded Documents

Embedded documents are **documents inside another document**. They are used to store **related data together**. They are best used when data is frequently accessed together

This reduces the need for joins and improves performance.

General Syntax:

```
{  
  field: {  
    innerField1: value1,  
    innerField2: value2,  
    ...  
  }  
}
```



```
}
```

Example:

```
{  
  contact: {  
    email: "abc@example.com",  
    phone: "03339227287"  
  }  
}
```

2 Querying Documents

MongoDB uses the `find()` method to retrieve documents from a collection.

It supports:

- **Filtering** (e.g. `WHERE` in SQL)
- **Projection** (selecting specific fields, e.g. `SELECT column1, column2` in SQL)

General Syntax:

```
db.collection.find(filter, projection)
```

Explanation:

- The first parameter `filter` is a document that defines the criteria (conditions) to filter out documents.
- The second parameter `projection` is also a document that specifies the fields to be included or excluded in the output.
- While calling the method, either one or both of the parameters may be skipped by:
 - Passing no arguments `find()`
 - Passing empty documents as arguments `find( {}, {} )`

in order to return all documents containing all the fields.

Examples:

```
// Return all documents from students collection
```



```
db.students.find()

// Return all documents (explicit empty filter)
db.students.find({})

// Return all documents with all fields (explicit filter + projection)
db.students.find({}, {})
```

2.1 Sorting

Sorting is used to arrange documents in **ascending or descending order** based on one or more fields.

Sorting is applied **before displaying results**. If a field is missing, MongoDB may place those documents first or last (depending on order). Sorting can also be done on **multiple fields**.

The sorting order for a field is specified by the following two values:

- **1**: Ascending order (lowest to highest)
- **-1**: Descending order (highest to lowest)

General Syntax:

```
db.collection.find(...).sort( { field: 1/-1, ... } );
```

Examples:

```
// Sort students by CGPA in ascending order
db.students.find().sort({ cgpa: 1 })

// Sort students by CGPA (highest first)
db.students.find().sort({ cgpa: -1 })

// Sort by CGPA (descending), then by name (ascending)
db.students.find().sort({ cgpa: -1, name: 1 })
```

2.2 Limiting

Limiting is used to restrict the number of documents returned. It is commonly used with **sorting for top/bottom results**



General Syntax:

```
db.collection.find(...).limit(n);
```

Examples:

```
// Return only first 3 students
db.students.find().limit(3)

// Get the student with highest CGPA
db.students.find().sort({ cgpa: -1 }).limit(1)
```

2.3 Skipping

Skipping is used to **ignore a number of documents** before returning results. It may also be combined/chained with sorting and/or limiting documents.

It is mainly used for **pagination**.

General Syntax:

```
db.collection.find(...).skip(n);
```

Examples:

```
// Skip first 2 students
db.students.find().skip(2)

// Skip 1 document, then return the next one
db.students.find().skip(1).limit(1)

// Get the 4th highest CGPA student
db.students.find().sort( { cgpa: -1 } ).skip(3).limit(1);
```

2.4 Projection

Projection controls which fields are returned in query results. It improves readability and performance by returning only required data. It is specified as the **second parameter of find()**.

Properties:



- The `_id` field is always returned by default. Other fields can be either included or excluded using boolean values (`true`, `false`) or `1` and `0` respectively.
- With the exception of the `_id` field, it is not allowed to combine inclusion and exclusion statements in projection documents.

General Syntax:

```
db.collection.find( ..., { field: true/false, ... } );
```

Examples:

```
// Show only name and CGPA (SQL: SELECT name, cgpa FROM students)
db.students.find( {}, { name: true, cgpa: true } );

// Show name and CGPA but hide _id
db.students.find({}, { _id: 0, name: 1, cgpa: 1 })

// Hide address field, show all others
db.students.find({}, { address: false })

// INVALID: Cannot mix inclusion and exclusion (except _id)
db.students.find({}, { name: 1, cgpa: 0 })
```

2.5 Filtering

Filtering is used to **retrieve documents based on conditions**. It works like the **WHERE** clause in SQL. It is specified as the **first parameter of `find()`**.

Multiple comma separated filter conditions are treated as implicit **AND**.

General Syntax:

```
db.collection.find( { field: value, ... }, ...)
```

Examples:

```
// Find all CS students
db.students.find({ department: "CS" })
```




```
// Multiple conditions (AND logic by default)
db.students.find({ department: "CS", cgpa: 3.3 })
```

2.5.1 Comparison Operators

Comparison operators are used to **compare field values** and filter documents based on numeric or exact conditions.

They are commonly used for numbers, strings, and filtering ranges.

Operator	Description
\$eq	Equal
\$ne	Not equal
\$gt	Greater than
\$lt	Less than
\$gte	Greater or equal
\$lte	Less or equal
\$exists	Checks if field exists

General Syntax:

```
db.collection.find( { field: { operator: value, ... } } )
```

Examples:

```
// Students with CGPA greater than 3.5
db.students.find({ cgpa: { $gt: 3.5 } })

// Students with age less than or equal to 20
db.students.find({ age: { $lte: 20 } })

// Students whose CGPA is NOT 3.2
db.students.find({ cgpa: { $ne: 3.2 } })
```



```
// Students whose age is 21
db.students.find({ age: { $eq: 21 } }) // same as { age: 21 }
```

2.5.2 Handling Missing Values

MongoDB allows documents to have missing fields or null values, which must be handled carefully during queries.

General Syntax:

```
// Check if field exists or not
db.collection.find({ field: { $exists: true/false } })

// Check if field is null OR missing
db.collection.find({ field: null })
```

Examples:

```
// Students who have CGPA field
db.students.find({ cgpa: { $exists: true } })

// Students where CGPA field is missing
db.students.find({ cgpa: { $exists: false } })

// Students with null or missing department
db.students.find({ department: null })
```

2.5.3 Logical Operators

Logical operators are used to combine multiple conditions.

Operator	Description
\$and	All conditions must be true
\$or	Any condition can be true
\$nor	All conditions must be false



\$not

Negates a condition

General Syntax:

```
db.collection.find({ operator: [ { field: val }, {field: val}, ...] }  
);  
  
db.collection.find( { field: { $not: { operator: value } } } );
```

Examples:

```
// Students from CS OR students with CGPA > 3.7  
db.students.find( {  
  $or: [ { department: "CS" }, { cgpa: { $gt: 3.7 } } ]  
} )  
  
// SE students with CGPA less than 3.0  
db.students.find( {  
  $and: [ { department: "SE" }, { cgpa: { $lt: 3.0 } } ]  
} )  
  
// Students who are NOT in CS or SE department  
db.students.find( {  
  $nor: [ { department: "CS" }, { department: "SE" } ]  
} )  
  
// Students whose CGPA is NOT greater than 3.5  
db.students.find(  
  cgpa: { $not: { $gt: 3.5 } }  
)
```

2.5.4 Array Filtering

Array filtering is used to **query documents based on values inside array fields**.

In MongoDB, arrays are treated as **first-class searchable fields**, meaning you can directly query inside them without joins.



2.5.4.1 Basic Array Matching

MongoDB automatically checks **inside arrays** when matching values. If a field is an array, MongoDB **checks each element**.

General Syntax:

```
db.collection.find({ field: value })
```

Examples:

```
// Find students who know Python (inside skills array)
db.students.find({ skills: "Python" })

// Students from CS department who know JavaScript
db.students.find({ department: "CS", skills: "JS" })
```

2.5.4.2 Array Operators

Array operators provide **advanced control over array queries**.

\$in and **\$nin** operators work on both arrays and normal fields.

Operator	Description
\$size	Matches arrays of exact size
\$all	All listed values should exist in the array
\$in	Any one of the listed values should occur in array / simple field
\$nin	None of the listed values should be present in the array / simple field

General Syntax:

```
db.collection.find( { field: { operator: value } } )
```

Examples:

```
// Students with exactly 2 skills
db.students.find({ skills: { $size: 2 } })
```



```
// Students who know BOTH Python and JS
db.students.find({ skills: { $all: ["Python", "JS"] } })

// Students who know Python OR C++
db.students.find({ skills: { $in: ["Python", "C++"] } })

// Students who do NOT know Java or PHP
db.students.find({ skills: { $nin: ["Java", "PHP"] } })
```

2.5.5 Querying Embedded Documents

Embedded documents (nested objects) are queried using **dot notation**, which allows access to fields inside sub-documents.

This is similar to accessing object properties in programming languages. It works with **all comparison and logical operators**.

General Syntax:

```
db.collection.find( { "outerField.innerField": value } )
```

Examples:

```
// Find students living in Lahore
db.students.find({ "address.city": "Lahore" })

// Find students with ZIP code 75010
db.students.find({ "address.zip": 75010 })

// Find students with ZIP code greater than 50000
db.students.find({ "address.zip": { $gt: 50000 } })

// Students from Lahore OR Karachi
db.students.find({
  $or: [
    { "address.city": "Peshawar" },
    { "address.city": "Karachi" }
  ]
})
```



National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology



})